

Example Mission Alpha

The first canonical MOOS-IvP example mission.

Included MIT MOOS-IvP PDF sources:

- chap_alpha.pdf

A First Example with MOOS-IvP - the Alpha Mission

Fall 2024

Michael Benjamin, mikerb@mit.edu
Department of Mechanical Engineering
MIT, Cambridge MA 02139
project-pavlab/chapters/chap_alpha

1	A First Example with MOOS-IvP - the Alpha Mission	1
1.1	Find and Launch the Alpha Example Mission	2
1.2	A Look at the Behavior File used in the Alpha Example Mission	4
1.3	A Closer Look at the MOOS Apps in the Alpha Example Mission	6

Contents

1 A First Example with MOOS-IvP - the Alpha Mission

This section describes a simple mission using the helm. It is designed to run in simulation on a single machine. The mission configuraiton files for this example are distributed with the source code. Information on how to find these files and launch this mission are described below in Section 1.1. In this example the vehicle simply traverses a set of pre-defined given waypoints and returns back to the launch position. The user may return the vehicle any time before completing the waypoints, and may subsequently command the vehicle to resume the waypoints at any time. This example touches on the following issues:

- Launching a mission with a given mission (.moos) file and behavior (.bhv) file.
- Configuration of MOOS processes, including the IvP Helm, with a .moos file.
- Configuration of the IvP Helm (mission planning) with a .bhv file.
- Implementation of simple command and control with the IvP Helm.
- Interaction between MOOS processes and the helm during normal mission operation.

Here is a bit of what the Alpha mission should look like:

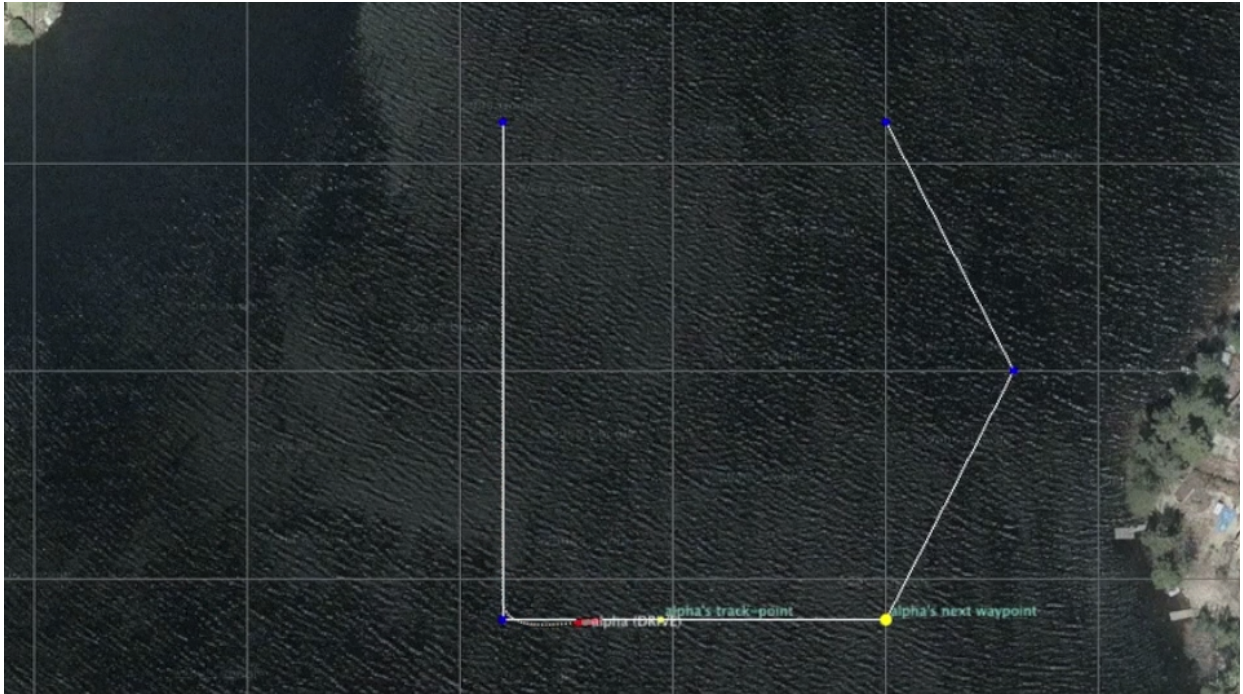


Figure 1: The Alpha mission.

video:(0:19): <https://vimeo.com/84549446>

1.1 Find and Launch the Alpha Example Mission

The example mission should be in the same directory tree containing the source code. There are two files - a MOOS file, also mission file or .moos file, and a behavior file or .bhv file:

```
moos-ivp/
  MOOS/
    ivp/
      missions/
        s1_alpha/
          alpha.moos  <---- The MOOS file
          alpha.bhv   <---- The Behavior file
```

To run this mission from a terminal window, simply change directories and launch:

```
$ cd moos-ivp/ivp/missions/s1_alpha
$ pAntler alpha.moos
```

After **pAntler** has launched each process, the **pMarineViewer** window should be open and look similar to that shown in Figure 2. After clicking the DEPLOY button in the lower right corner the vehicle should start to traverse the shown set of waypoints.

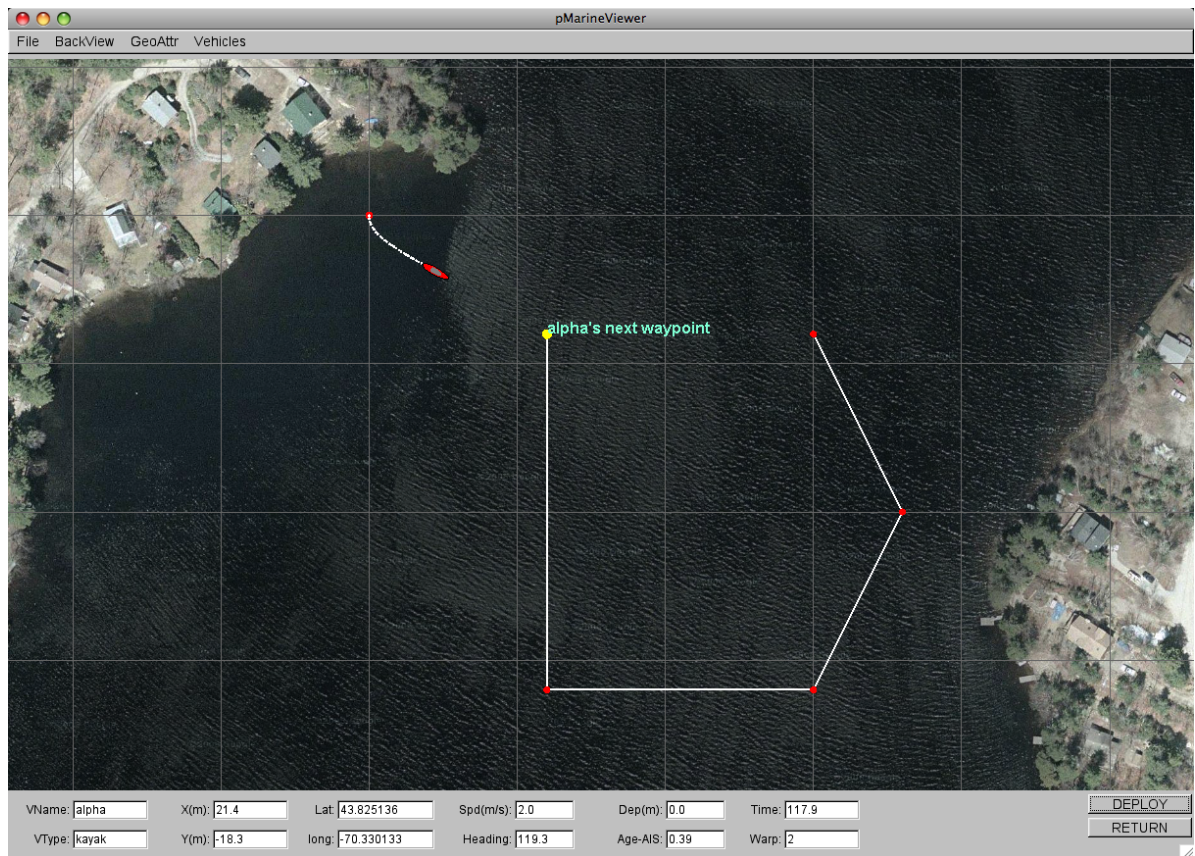


Figure 2: **The Alpha Example Mission - In the Surveying Mode:** A single vehicle is dispatched to traverse a set of waypoints and, upon completion, traverse to the waypoint (0,0) which is the launch point.

This mission will complete on its own with the vehicle returning to the launch point. Alternatively, by hitting the **RETURN** button at any time before the points have been traverse, the vehicle will change course immediately to return to the launch point, as shown in Figure 3. When the vehicle is returning as in the figure, it can be re-deployed by hitting the **DEPLOY** button again.

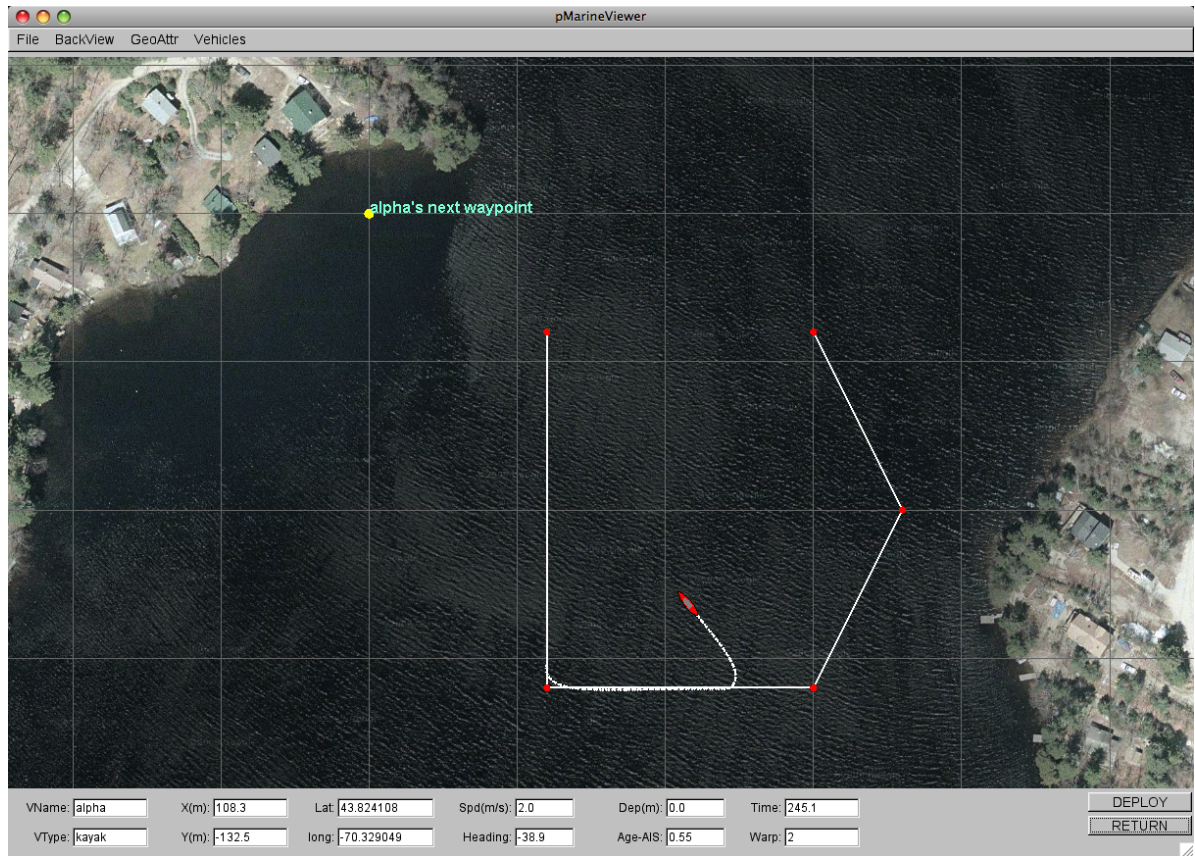


Figure 3: **The Alpha Example Mission - In the Returning Mode**: The vehicle can be commanded to return prior to the completion of its waypoints by the user clicking the **RETURN** button on the viewer.

The vehicle in this example is configured with two basic waypoint behaviors. Their configuration with respect to the points traversed and when each behavior is actively influencing the vehicle, is discussed next.

1.2 A Look at the Behavior File used in the Alpha Example Mission

The mission configuration of the helm behaviors is provided in a *behavior file*, and the complete behavior file for the example mission is shown in Listing 1. Behaviors are configured in blocks of parameter-value pairs - for example lines 6-17 configure the waypoint behavior with the five waypoints shown in the previous two figures. This is discussed in more detail in Section ??.

Listing 1.1: The behavior file for the Alpha example.

```

1 initialize  DEPLOY = false
2 initialize  RETURN = false
3
4 //-----
5 Behavior = BHV_Waypoint
6 {
7   name      = waypt_survey
8   pwt       = 100

```

```

 9   condition = RETURN = false
10   condition = DEPLOY = true
11   endflag   = RETURN = true
12   perpetual = true
13
14       lead = 8
15   lead_damper = 1
16       speed = 2.0    // meters per second
17       radius = 4.0
18   nm_radius = 10.0
19       points = 60,-40:60,-160:150,-160:180,-100:150,-40
20       repeat = 1
21 }
22
23 //-----
24 Behavior = BHV_Waypoint
25 {
26     name      = waypt_return
27     pwt       = 100
28     condition = RETURN = true
29     condition = DEPLOY = true
30     perpetual = true
31     endflag   = RETURN = false
32     endflag   = DEPLOY = false
33
34     speed = 2.0
35     radius = 2.0
36     nm_radius = 8.0
37     point = 0,0
38 }

```

The parameters for each behavior are separated into two groups. Parameters such as **name**, **priority**, **condition** and **endflag** are parameters defined generally for all IvP behaviors. Parameters such as **speed**, **radius**, and **points** are defined specifically for the Waypoint behavior. A convention used in .bhv files is to group the general behavior parameters separately at the top of the configuration block.

In this mission, the vehicle follows two sets of waypoints in succession by configuring two instances of a basic waypoint behavior. The second waypoint behavior (lines 23-37) contains only a single waypoint representing the vehicle launch point (0,0). It's often convenient to have the vehicle return home when the mission is completed - in this case when the first waypoint behavior has reached its last waypoint. Although it's possible to simply add (0,0) as the last waypoint of the first waypoint behavior, it is useful to keep it separate to facilitate recalling the vehicle pre-maturely at any point after deployment.

Behavior conditions (lines 9-10, 28-29), and endflags (line 110, lines 31-32) are primary tools for coordinating separate behaviors into a particular mission. Behaviors will not participate unless each of its conditions are met. The conditions are based on current values of the MOOS variables involved in the condition. For example, both behaviors will remain idle unless the variable **DEPLOY** is set to true. This variable is set initially to be false by the initialization on line 1, and is toggled by the DEPLOY button on the **pMarineViewer** GUI shown in Figures 2 and 3. The **pMarineViewer** MOOS application is one option for a command and control interface to the helm. The MOOS variables in the behavior conditions in Listing 1 do not care which process was responsible for setting the value. Endflags are used by behaviors to post a MOOS variable and value when a behavior has reached a completion. The notion of completion is different for each behavior and some behaviors have no

notion of completion, but in the case of the waypoint behavior, completion is declared when the last waypoint is reached. In this way, behaviors can be configured to run in a sequence, as in this example, where the returning waypoint behavior will have a necessary condition (line 28) met when the surveying behavior posts its endflag on line 11.

1.3 A Closer Look at the MOOS Apps in the Alpha Example Mission

Running the example mission involves five other MOOS applications in addition to the IvP helm. In this section we take a closer look at what those applications do and how they are configured. The full MOOS file, `alpha.moos`, used to run this mission is given in full in the appendix. An overview of the situation is shown in Figure 4.

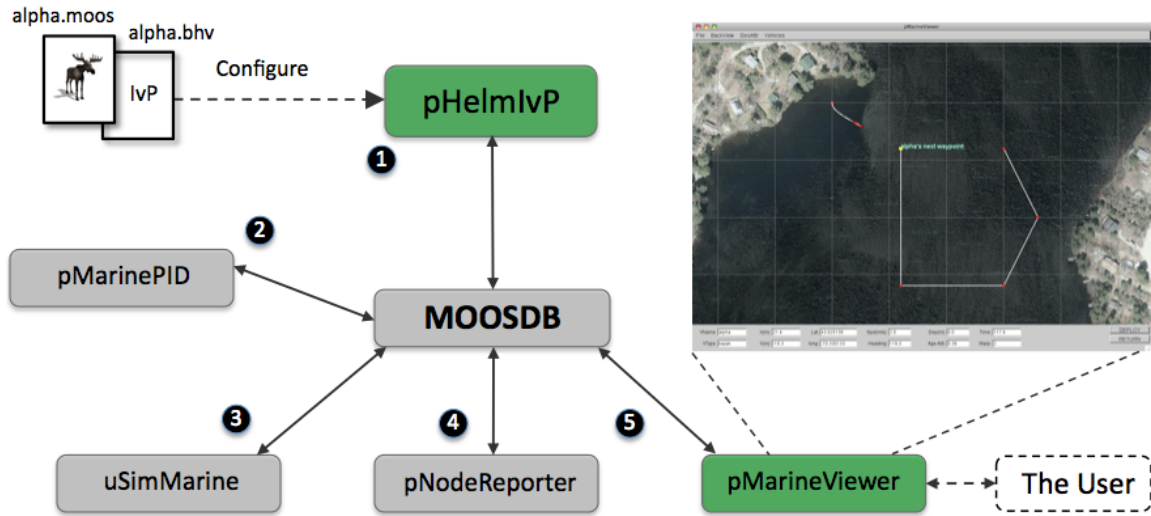


Figure 4: **The MOOS processes in the example "alpha" mission:** In (1) The helm produces a desired heading and speed. In (2) the PID controller subscribes for the desired heading and speed and publishes actuation values. In (3) the simulator grabs the actuator values and the current vehicle pose and publishes a set of MOOS variables representing the new vehicle pose. In (4) all navigation output is wrapped into a single node-report string to be consumed by the helm and the GUI viewer. In (5) the pMarineViewer grabs the node-report and renders a new vehicle position. The user can interact with the viewer to write limited command and control variables to the MOOSDB.

1.3.1 Antler and the Antler Configuration Block

The `pAntler` tool is used to orchestrate the launching of all the MOOS processes participating in this example. From the command line, `pAntler` is run with a single argument the `.moos` file. As it launches processes, it hands each process a pointer to this same MOOS file. The Antler configuration block in this example looks like:

Listing 1.2: An example Antler configuration block for the Alpha mission.

```

1 ProcessConfig = ANTLER
2 {
3   MSBetweenLaunches = 200
4 }

```

```

5   Run = MOOSDB           @ NewConsole = false
6   Run = uSimMarine       @ NewConsole = false
7   Run = pNodeReporter    @ NewConsole = false
8   Run = pMarinePID       @ NewConsole = false
9   Run = pMarineViewer    @ NewConsole = false
10  Run = pHelmIvP         @ NewConsole = false
11 }

```

The first parameter on line 2 specifies how much time should be left between the launching of each process. Lines 4-9 specify which processes to launch. The MOOSDB is typically launched first. The `NewConsole` switch on each line determines whether a new console window should be opened with each process. Try switching one or more of these to `true` as an experiment.

1.3.2 The pMarinePID Application

The `pMarinePID` application implements a simple PID controller which produces values suitable for actuator control based on inputs from the helm. In simulation the output is consumed by the vehicle simulator rather than the vehicle actuators.

In short: The `pMarinePID` application typically gets its info from `pHelmIvP`; produces info consumed by `uSimMarine` or actuator MOOS processes when not running in simulation.

Subscribes to: `DESIRED_HEADING`, `DESIRED_SPEED`.

Publishes to: `DESIRED_RUDDER`, `DESIRED_THRUST`.

1.3.3 The uSimMarine Application and Configuration Block

The `uSimMarine` application is a very simple vehicle simulator that considers the current vehicle pose and actuator commands and produces a new vehicle pose. It can be initialized with a given pose as shown in the configuration block used in this example, shown in Listing 3:

Listing 1.3: An example uSimMarine configuration block for the Alpha mission.

```

1  ProcessConfig = uSimMarine
2  {
3      AppTick    = 10
4      CommsTick  = 10
5
6      START_X     = 0
7      START_Y     = 0
8      START_SPEED = 0
9      START_HEADING = 180
10     PREFIX      = NAV
11 }

```

In short: The `uSimMarine` application typically gets its info from `pMarinePID`; produces info consumed by `pNodeReporter` and itself on the next iteration of `uSimMarine`.

Subscribes to: `DESIRED_RUDDER`, `DESIRED_THRUST`, `NAV_X`, `NAV_Y`, `NAV_SPEED`, `NAV_HEADING`.

Publishes to: `NAV_X`, `NAV_Y`, `NAV_HEADING`, `NAV_SPEED`.

1.3.4 The pNodeReporter Application and Configuration Block

An Automated Information System (AIS) is commonplace on many larger marine vessels and is comprised of a transponder and receiver that broadcasts one's own vehicle ID and pose to other nearby vessels equipped with an AIS receiver. It periodically collects all latest pose elements, e.g., latitude and longitude position and latest measured heading and speed, and wraps it up into a single update to be broadcast. This MOOS process collects pose information by subscribing to the MOOSDB for NAV_X, NAV_Y, NAV_HEADING, NAV_SPEED, and NAV_DEPTH and wraps it up into a single MOOS variable called NODE_REPORT_LOCAL. This variable in turn can be subscribed to another MOOS process connected to an actual serial device acting as an AIS transponder. For our purposes, this variable is also subscribed to by pMarineViewer for rendering a vehicle pose sequence.

In short: The pNodeReporter application typically gets its info from uSimMarine or otherwise on-board navigation systems such as GPS or compass; produces info consumed by pMarineViewer and instances of pHelmIvP running in other vehicles or simulated vehicles.

Subscribes to: NAV_X, NAV_Y, NAV_SPEED, NAV_HEADING.

Publishes to: NODE_REPORT_LOCAL

1.3.5 The pMarineViewer Application and Configuration Block

The pMarineViewer is a MOOS process that subscribes to the MOOS variable NODE_REPORT_LOCAL and NODE_REPORT which contains a vehicle ID, pose and timestamp. It renders the updated vehicle(s) position. It is a multi-threaded process to allow both communication with MOOS and let the user pan and zoom and otherwise interact with the GUI. It is non-essential for vehicle operation, but essential for visually confirming that all is going as planned.

In short: The pMarineViewer application typically gets its info from pNodeReporter and pHelmIvP; produces info consumed by pHelmIvP when configured to have command and control hooks (as in this example).

Subscribes to: NODE_REPORT, NODE_REPORT_LOCAL, VIEW_POINT, VIEW_SEGLIST, VIEW_POLYGON, VIEW_MARKER.

Publishes to: Depends on configuration, but in this example: DEPLOY, RETURN.